

Tema 4 — Árboles de Decisión y Ensamblados

PARTE 1 — ÁRBOLES DE DECISIÓN

1. ¿QUÉ ES UN ÁRBOL DE DECISIÓN?

Una estructura que divide los datos en ramas basadas en decisiones (condiciones) que conducen a una predicción final (una clase o un valor numérico). Es una técnica de aprendizaje supervisado que puede aplicarse tanto a problemas de **clasificación** como de **regresión**.

Componentes:

- **Nodo raíz:** el nodo principal, en la parte superior
- **Nodos internos:** representan preguntas/condiciones sobre un atributo
- **Nodos terminales (hojas):** donde termina el flujo; contienen la respuesta final (la clasificación)

Un **nodo de decisión** está asociado a un atributo y tiene 2 o más ramas que salen de él, cada una representando los posibles valores que puede tomar ese atributo.

2. ID3 (Iterative Dichotomiser 3)

Creado por Ross Quinlan. Genera un árbol de decisión a partir de un conjunto de ejemplos, usando **Entropía** y **Ganancia de Información** para decidir qué atributo usar en cada nodo.

Entropía (de la información)

Es la medida del desorden, o la medida de la pureza/impureza de los datos.

$$\text{Entropía}(S) = -\sum p_i * \log_2(p_i)$$

Donde:

- S: lista de valores posibles
- p_i : probabilidad de cada valor
- i: cada uno de los valores

Puntos clave:

- Para una muestra homogénea (todos los ejemplos son de la misma clase), la entropía es **0** (no hay desorden, está "pura")
- La máxima entropía es $\log_2(n)$, donde n es la cantidad de valores de salida posibles. Si $n=2$ (por ejemplo TRUE/FALSE), la máxima entropía es **1** — la máxima incertidumbre

posible

Ganancia de Información

Cuantifica qué característica, de un conjunto de datos, proporciona la máxima información sobre la clasificación. Sirve para elegir qué atributo conviene usar en cada nodo.

$$\text{Ganancia}(S,A) = \text{Entropía}(S) - \sum (|S_v|/|S|) * \text{Entropía}(S_v)$$

Donde:

- S: lista de valores posibles para el atributo A
- A: el atributo evaluado
- V(A): conjunto de valores que A puede tomar
- S_v/S : probabilidad de un valor para el atributo A
- Entropía(S_v): entropía calculada para el valor "v" de A

Los 3 pasos básicos de ID3, explicados uno por uno:

Paso 1 — Calcular la entropía del conjunto de datos completo

Se calcula UN SOLO número: la entropía de todos los ejemplos juntos, antes de dividir por ningún atributo. La fórmula usa un término por cada clase posible (por eso a veces se lo describe como "calcular la entropía de todas las clases"), pero el resultado es un único valor de entropía para todo el conjunto S. Este valor es el "punto de partida" contra el que se va a comparar todo lo demás.

Paso 2 — Calcular la entropía de cada subgrupo, para cada atributo candidato

Para CADA atributo que se quiera evaluar (por ejemplo "¿tiene plumas?", "¿pone huevos?"), se divide el conjunto de datos según los valores que puede tomar ese atributo, y se calcula la entropía de CADA subgrupo resultante por separado.

La cantidad de entropías que se obtienen en este paso depende de cuántos valores puede tomar ese atributo en particular: un atributo binario (Sí/No) da 2 entropías; un atributo con 3 categorías da 3 entropías. Es la MISMA fórmula de entropía del Paso 1, solo que aplicada varias veces, una por cada subgrupo, en vez de una sola vez sobre todo el conjunto.

Paso 3 — Elegir el atributo que da la mayor Ganancia de Información

Con los resultados de los Pasos 1 y 2, se calcula la Ganancia de Información de cada atributo (Entropía de antes menos el promedio ponderado de las entropías de después). Se compara la Ganancia de todos los atributos candidatos, y se elige el que tenga la Ganancia MÁS ALTA — ese es el atributo que reduce más el desorden, y se usa para ese nodo del árbol (el nodo raíz, si es la primera división; un nodo interno, si ya se venía dividiendo antes).

Después, el proceso se repite dentro de cada rama que haya quedado con entropía mayor a 0 (todavía mezclada), usando solo los datos de esa rama y los atributos que no se hayan usado

todavía en ese camino, hasta que cada rama termine en un nodo puro (entropía 0) o se acaben los atributos disponibles.

Ejemplo numérico (de las diapos):

```
Entropía inicial del conjunto: 1.56

Para el atributo "Vuela":
  p(Sí) = 0.71, Entropía(Sí) = 1.52
  p(No) = 0.29, Entropía(No) = 1

Ganancia = 1.56 - (0.29*1 + 0.71*1.52) = 0.1908
```

3. DOS CAMINOS ALTERNATIVOS PARA ELEGIR EL ATRIBUTO: ENTROPÍA/GANANCIA vs GINI

Es importante no mezclar estos dos métodos: son dos caminos **alternativos** para el mismo objetivo (decidir qué atributo usar en cada nodo), no un método que se calcula "a partir" del otro. Un árbol usa UNO de los dos caminos, no ambos combinados.

Camino A — Entropía y Ganancia de Información (usado por ID3):

```
Entropía del conjunto → Entropía de cada subgrupo → Ganancia de Información
                                                    ↓
                        Se elige el atributo con MAYOR ganancia
```

Camino B — Impureza de Gini (usado por otras implementaciones, como CART):

```
Impureza de Gini de cada subgrupo → promedio ponderado (Gini total del atributo)
                                                    ↓
                        Se elige el atributo con MENOR impureza de Gini
```

Tabla comparativa:

	Entropía / Ganancia de Información	Impureza de Gini
Fórmula base	$-\sum p_i \cdot \log_2(p_i)$	$1 - \sum p_i^2$
Qué se calcula para decidir	Ganancia (entropía antes – entropía ponderada después)	Impureza ponderada de cada atributo, directamente
Se elige el atributo con...	Mayor ganancia	Menor impureza

4. IMPUREZA DE GINI (detalle del Camino B)

Mide cuán a menudo un elemento elegido al azar del conjunto sería etiquetado incorrectamente si se etiquetara al azar según la distribución de las etiquetas del subconjunto.

Pasos del Camino B, en detalle:

1. Se calcula la impureza de Gini para cada nodo hoja resultante de dividir por cada atributo candidato
2. Se calcula el promedio ponderado de esas impurezas → la impureza de Gini total del atributo
3. Se elige el atributo con **menor** impureza de Gini total (menor impureza = mejor separación de las clases)

Un nodo es "**puro**" cuando el 100% de sus casos pertenece a una sola clase. Si no es 100% puro, es un nodo impuro y hace falta seguir dividiendo o aceptar el error.

5. C4.5 — MEJORAS SOBRE ID3

Se hicieron 3 mejoras principales sobre ID3:

a) Campos numéricos / rangos continuos

Si un atributo A tiene un rango continuo de valores, el algoritmo puede crear dinámicamente un campo booleano: si $A < C$ entonces $A_c = \text{TRUE}$, sino $A_c = \text{FALSE}$.

¿Cómo se encuentra el umbral C? Se ordena A de menor a mayor, se identifican los puntos donde cambia el valor de la clase de salida (ahí están los candidatos a C), se prueban varios C candidatos y se elige el que da mayor Ganancia de Información.

b) Datos faltantes

C4.5 permite que los valores de un atributo se marquen como "?" cuando faltan. Esos valores faltantes simplemente NO se usan en los cálculos de ganancia y entropía (no hace falta imputarlos previamente).

c) Poda (Pruning)

El método de poda consiste en:

1. Generar el árbol completo, tal como se vio antes
2. Analizar recursivamente, desde las hojas hacia arriba, qué nodos internos se pueden eliminar sin que aumente el error de clasificación sobre el conjunto de test
3. Se elimina un nodo interior cuyos sucesores son todos nodos hoja
4. Se recalcula el error sobre el conjunto de test con el nuevo árbol (más chico)

5. Si el error es menor o igual que antes, se mantiene la poda (se elimina el nodo y sus sucesores)
6. Se repite el proceso

La poda existe específicamente para evitar el OVERFITTING — un árbol completo sin podar tiende a memorizar ruido del conjunto de entrenamiento.

PARTE 2 — ENSAMBLES

6. ¿QUÉ ES UN ENSAMBLE?

Combinar varios modelos (llamados "weak learners" o estimadores débiles) para lograr un modelo final más robusto que cualquiera de los modelos individuales. La idea intuitiva: "muchos estimadores mediocres, promediados, pueden ser muy buenos".

Amount of say (peso de voto): es la importancia que se le da a cada clasificador base dentro del modelo final. Se asigna en función de la precisión de ese clasificador — cuanto mejor rinde, más peso tiene en la votación final.

Dos grandes categorías:

- **Ensamblés Homogéneos:** todos los modelos base son del mismo tipo (por ejemplo, todos árboles) → Bagging, Random Forest, Boosting
 - **Ensamblés Híbridos:** combinan modelos de tipos DIFERENTES → Voting, Stacking, Cascading
-

7. BAGGING (Bootstrap Aggregating)

Se refiere al **muestreo con reemplazo** de los datos para entrenar múltiples modelos **en paralelo** (no secuencial, a diferencia de boosting).

¿Cómo funciona (Bootstrap Aggregating)?

Dado un conjunto de entrenamiento D de tamaño n , se generan m nuevos conjuntos de entrenamiento $D_1, D_2, \dots, D_m$, cada uno de tamaño n' , tomando muestras aleatorias **CON REEMPLAZO** de D . Generalmente n' es aproximadamente $2/3$ de n .

Ejemplo numérico (de las diapos):

Dataset original: 9000 registros, 8 atributos, 1 clase

Se toman m sub-tablas de 6000 filas cada una ($2/3$ de 9000), seleccionadas aleatoriamente **CON reemplazo**.

Attribute Bagging (Random Subspace): además de tomar filas al azar, para CADA una de las m tablas se toman también solo algunos ATRIBUTOS (columnas) al azar. La cantidad de atributos a tomar es la **raíz cuadrada** del número total de atributos.

Ejemplo: 8 atributos totales $\rightarrow \text{Round}(\sqrt{8}) = 3$ atributos por sub-tabla

Esto es especialmente recomendable cuando hay muchos atributos.

8. RANDOM FOREST

Es un algoritmo de ensamble basado en árboles de decisión, que usa Bagging + Attribute Bagging. Combina varios árboles de decisión para mejorar la precisión y reducir el riesgo de overfitting.

Proceso completo:

1. Generar m sub-tablas con Bootstrap (filas aleatorias con reemplazo)
2. Para cada sub-tabla, quedarse solo con $\sqrt{\text{cantidad de atributos}}$ columnas al azar
3. Entrenar un árbol de decisión con cada una de las m tablas reducidas
4. Para cada árbol, calcular su matriz de confusión y su **tasa de error**: (Falsos Positivos + Falsos Negativos) / Total de ejemplos clasificados
5. Quedarse con el mejor árbol de los m , y repetir el proceso K veces
6. Al final se tienen K árboles (el "bosque")

¿Cómo clasificar un nuevo ejemplo?

Se ejecuta el ejemplo a clasificar por cada uno de los K árboles. Si la mayoría de los árboles devuelve "CATEGORÍA-A", esa es la predicción final (votación por mayoría).

Random Forest usa BAGGING (entrena en paralelo, cada árbol independiente del resto), a diferencia de Boosting.

9. BOOSTING

A diferencia de Bagging, en Boosting los modelos se entrenan de forma **secuencial**, no en paralelo. Cada modelo se entrena sobre TODO el conjunto de entrenamiento, pero el truco está en que cada nuevo modelo se enfoca en corregir los errores del modelo anterior — las instancias mal clasificadas obtienen **mayor peso** en el entrenamiento del siguiente modelo.

Diferencia clave con Bagging:

	Bagging	Boosting
Entrenamiento	Paralelo (independiente)	Secuencial (cada modelo depende del anterior)
Enfoque	Muestras distintas al azar	Todo el dataset, pero con pesos ajustados
Objetivo principal	Reducir VARIANZA	Reducir SESGO (bias)
Ejemplo	Random Forest	AdaBoost, Gradient Boost, XGBoost

10. ADABOOST

La técnica consiste en entrenar un clasificador base (por ejemplo un árbol de decisión simple), verificar los errores que comete, y entrenar luego otro clasificador que corrija esos errores. Se repite este proceso hasta disminuir el error o encontrar un clasificador perfecto.

Mecanismo: usa el mismo conjunto de entrenamiento en cada iteración, pero antes de entrenar el siguiente predictor, **pondera** (aumenta el peso relativo de) las instancias mal clasificadas. Así, el siguiente predictor se enfoca en corregir los errores del anterior.

Diferencia con Random Forest: en Random Forest hay N árboles COMPLETOS (de distinta profundidad). En AdaBoost hay un **bosque de tocones (stumps)** — árboles de un solo nivel (una sola pregunta).

Resumen de AdaBoost:

1. Combina muchos weak learners (generalmente stumps) para clasificar
2. Algunos stumps tienen más peso que otros en la votación final ("amount of say")
3. Cada stump se construye teniendo en cuenta los errores del stump anterior (usando Gini ponderado, o regenerando los datos con más peso en los errores)

11. GRADIENT BOOST

Se centra en minimizar una función de pérdida utilizando el **gradiente del error** de los modelos previos (de ahí el nombre).

Mecanismo:

- Crea una cadena de árboles de profundidad fija (limitada, no completos)
- Comienza con un único valor (un nodo hoja, la predicción inicial)
- Luego calcula árboles sucesivos, cada uno prediciendo el ERROR (residuo) cometido por el modelo anterior
- Cada árbol está ponderado por un factor constante llamado **Learning Rate** (tasa de aprendizaje, valor entre 0 y 1)

- Los árboles están restringidos en su crecimiento (profundidad limitada)

¿Por qué se necesita el Learning Rate? Si no se usara, el modelo predeciría perfectamente sobre el conjunto de entrenamiento (100% overfitting), porque la predicción sería exactamente el valor real. El Learning Rate limita cuánto se corrige en cada paso, dejando margen para que sigan aportando los árboles siguientes y mejorando la generalización.

Se siguen agregando árboles hasta que los residuos no cambien significativamente, o hasta alcanzar un número preestablecido de árboles.

Predicción final: valor inicial + suma de los residuos ponderados de cada árbol (cada uno multiplicado por el learning rate).

12. XGBOOST (eXtreme Gradient Boost)

Es una mejora avanzada de Gradient Boosting, optimizada para velocidad y eficiencia (diseñado para Big Data). Incluye regularización para reducir overfitting, soporte para manejo de valores nulos, y procesamiento paralelo.

Mecanismo paso a paso (con ejemplo de regresión):

1. **Predicción inicial:** por defecto se toma 0.5 (mismo valor sea clasificación o regresión)
2. **Construir un árbol para los residuos:** se calculan los residuos (diferencia entre valor real y la predicción actual) y se ponen todos en un nodo hoja inicial
3. **Calcular el Similarity Score:**

$$\text{Similarity Score} = (\text{Suma de residuos})^2 / (\text{Cantidad de residuos} + \lambda)$$

Donde λ (lambda) es un parámetro de **regularización**.

4. **Elegir el mejor split:** se calcula la Ganancia (Gain) para cada posible forma de dividir el árbol:

$$\text{Gain} = \text{Similarity}(\text{Nodo Izq}) + \text{Similarity}(\text{Nodo Der}) - \text{Similarity}(\text{Nodo Raíz})$$

Se elige la división que da mayor Gain.

5. **Repetir** hasta alcanzar la profundidad máxima estipulada
6. **Poda:** se elige un valor arbitrario llamado **gamma** (γ). Se calcula $(\text{Gain} - \gamma)$ para el nodo más bajo del árbol. Si esa diferencia es negativa, se poda ese nodo (y se sigue evaluando hacia arriba). Si es positiva, se conserva.
7. **Efecto de λ :** con $\lambda > 0$, los Similarity Scores (y por lo tanto los Gains) son menores $\rightarrow$ es más probable que haya que podar $\rightarrow \lambda$ previene el sobreentrenamiento/overfitting.
8. **Calcular el output de cada hoja:**

$$\text{Output} = (\text{Suma de residuos}) / (\text{Número de residuos} + \lambda)$$

9. Actualizar la predicción usando el Learning Rate (ϵ):

$$\text{Nueva predicción} = \text{Predicción anterior} + (\text{Learning Rate} * \text{Output del árbol})$$

Por defecto, el Learning Rate en XGBoost es 0.3.

10. **Repetir todo el proceso** (calcular nuevos residuos, construir nuevo árbol) tantas veces como iteraciones se hayan definido.

13. DIFERENCIAS ENTRE LOS 3 BOOSTS (muy preguntado)

AdaBoost: Comienza con un clasificador base simple. En cada iteración, ajusta los PESOS DE LAS MUESTRAS, aumentando el peso de las mal clasificadas. El modelo final es una combinación ponderada de todos los clasificadores, donde los que tuvieron mejor precisión tienen más peso ("amount of say") en la predicción.

Gradient Boosting: En vez de ajustar pesos de muestras, minimiza el error de un modelo base ajustando el siguiente modelo en dirección al **gradiente negativo de la función de pérdida**. Cada nuevo modelo intenta corregir los errores RESIDUALES del modelo anterior.

XGBoost: Similar a Gradient Boosting, pero optimizado en velocidad y precisión. Incorpora regularización (λ y γ), manejo de valores nulos y procesamiento paralelo, usando una técnica de búsqueda de nodos optimizada para minimizar el error en cada paso — más eficiente y menos propenso al overfitting que Gradient Boosting estándar.

14. ENSAMBLES HÍBRIDOS

A diferencia de los ensambles homogéneos (Bagging, Boosting), los híbridos pueden combinar modelos DIFERENTES entre sí (por ejemplo, Árboles + Naive Bayes + SVM juntos).

Voting

Varios modelos "votan" por el resultado final.

- En **clasificación**: funciona por mayoría de votos
- En **regresión**: funciona por promedio de las predicciones

Puede usar modelos de diferentes tipos, no solo basados en árboles como Random Forest.

Stacking

Los modelos base (que pueden ser diferentes entre sí) generan predicciones, y un **modelo final (meta-modelo)** combina esas predicciones para generar el resultado final.

Es más flexible que Voting, ya que puede usar una variedad de modelos — suelen usarse Árboles, Naive Bayes, SVM o Perceptrón como modelos base.

Diferencia clave con Voting: Voting combina las predicciones con una regla fija (mayoría/promedio); Stacking usa OTRO MODELO (el meta-modelo) que aprende cómo combinar mejor las predicciones de los modelos base.

Cascading

Los modelos se entrenan secuencialmente, pero NO como en Boosting. En Cascading, el modelo siguiente en la secuencia toma las **predicciones del modelo anterior como entrada adicional**, junto con las características originales — es un encadenamiento de modelos, pero sin el enfoque de "corregir errores" que tiene Boosting.

PREGUNTAS DE PRÁCTICA (formato final real)

Pregunta 1 (V/F — aparece en el ejemplo de parcial)

Indicar si las siguientes afirmaciones sobre árboles de decisión son verdaderas o falsas:

- a) Un árbol ID3 se construye utilizando la fórmula de entropía de la información, mientras que un árbol C4.5 se construye utilizando la impureza de Gini.
- b) En un Random Forest tradicional tengo un bosque de tocones (Stumps).
- c) Los árboles sólo pueden manejar variables numéricas normalizadas.
- d) La técnica de poda se utiliza en los árboles C4.5 para evitar el underfitting.

Pregunta 2 (Desarrollo)

Explique la diferencia entre Bagging y Boosting. ¿Cuál reduce varianza y cuál reduce sesgo? Dé un ejemplo de algoritmo de cada tipo.

Pregunta 3 (Desarrollo)

Explique las diferencias entre AdaBoost, Gradient Boosting y XGBoost.

Pregunta 4 (Desarrollo)

¿Qué diferencia hay entre Voting y Stacking? ¿Cuál es más flexible y por qué?

Pregunta 5 (Cálculo — Entropía)

Un conjunto de datos tiene 10 ejemplos: 6 de clase "Sí" y 4 de clase "No". Calcule la entropía de este conjunto.

RESPUESTAS MODELO

Respuesta 1

a) **FALSO.** Es al revés en cuanto a lo que dice la afirmación sobre las fórmulas específicas: ID3 usa Entropía y Ganancia de Información para elegir los atributos. C4.5 es una mejora de ID3 (agrega manejo de campos numéricos, datos faltantes y poda), y puede usar tanto Ganancia de Información como Impureza de Gini según la implementación — la afirmación tal como está planteada (que C4.5 usa EXCLUSIVAMENTE Gini) no es precisa; Gini es más asociado a CART, otro algoritmo de árboles.

b) **FALSO.** Un bosque de tocones (stumps) es característico de AdaBoost, no de Random Forest. En Random Forest tradicional los árboles son completos (de distinta profundidad cada uno), no simples tocones de un solo nivel.

c) **FALSO.** Los árboles de decisión pueden manejar variables categóricas directamente, y C4.5 específicamente permite manejar variables numéricas continuas creando umbrales dinámicos — no requieren normalización previa como sí necesitan otros algoritmos (por ejemplo KNN o redes neuronales).

d) **FALSO.** La poda en C4.5 se utiliza para evitar el OVERFITTING (sobreajuste), no el underfitting. Al eliminar nodos que no reducen el error de clasificación en el conjunto de test, se simplifica el árbol y se evita que memorice ruido de los datos de entrenamiento.

Respuesta 2

Bagging (Bootstrap Aggregating): entrena múltiples modelos EN PARALELO, cada uno con una muestra distinta (con reemplazo) del conjunto de entrenamiento. Como los modelos son independientes entre sí, promediar sus predicciones reduce la **VARIANZA** del modelo final. Ejemplo: Random Forest.

Boosting: entrena modelos de forma SECUENCIAL, donde cada modelo nuevo se enfoca en corregir los errores del modelo anterior (dándole más peso a las instancias mal clasificadas). Como cada modelo mejora iterativamente sobre el error del anterior, Boosting reduce principalmente el **SESGO (bias)**. Ejemplo: AdaBoost, Gradient Boosting, XGBoost.

Respuesta 3

AdaBoost: ajusta los PESOS DE LAS MUESTRAS en cada iteración, aumentando el peso de las mal clasificadas. Usa "bosques de tocones" (stumps). El modelo final combina los clasificadores ponderados según su precisión (amount of say).

Gradient Boosting: en vez de ajustar pesos de muestras, minimiza el error ajustando cada nuevo modelo en dirección al gradiente negativo de la función de pérdida. Cada árbol nuevo predice el residuo (error) del modelo anterior. Usa un Learning Rate para evitar el overfitting total.

XGBoost: una versión optimizada de Gradient Boosting. Agrega regularización explícita (parámetros λ y γ) para controlar el overfitting, maneja valores nulos automáticamente,

permite procesamiento paralelo, y usa el Similarity Score para decidir divisiones de forma más eficiente. Es más rápido y menos propenso a overfitting que Gradient Boosting estándar.

Respuesta 4

Voting combina las predicciones de varios modelos usando una regla FIJA y simple: mayoría de votos en clasificación, o promedio en regresión.

Stacking, en cambio, usa un MODELO ADICIONAL (el meta-modelo) que aprende cómo combinar de la mejor manera posible las predicciones de los modelos base — no usa una regla fija, sino que esa combinación también se entrena.

Stacking es más flexible porque el meta-modelo puede aprender patrones más complejos sobre cómo pesar o combinar las predicciones de cada modelo base, mientras que Voting siempre aplica la misma regla simple sin importar el contexto.

Respuesta 5

$$p(\text{Sí}) = 6/10 = 0.6$$

$$p(\text{No}) = 4/10 = 0.4$$

$$\begin{aligned}\text{Entropía} &= -\sum p_i * \log_2(p_i) \\ &= -(0.6 * \log_2(0.6)) - (0.4 * \log_2(0.4)) \\ &= -(0.6 * -0.737) - (0.4 * -1.322) \\ &= 0.442 + 0.529 \\ &= 0.971\end{aligned}$$

La entropía es 0.971, cercana al máximo (1 para 2 clases), lo que indica que el conjunto está bastante mezclado/impuro (no muy lejos de una distribución 50/50).